Smart Interview

Final Project Report

NLP / LLM Course Project

Daniel Shatzov and Shai Gigi

Dataset version: official_v1

Repository: Daniel23sh/NLP_Course

Report generated from committed project artifacts and final visualization outputs.

Executive Summary

This project defines and evaluates a fine-grained NLP task for scoring junior developer interview answers. Given a free-text answer about a project experience, the system predicts six ordinal rubric scores from 1 to 5: technical depth, personal contribution, clarity, problem solving, impact, and role relevance. The task also derives weak aspects (scores <= 2) and strong aspects (scores >= 4).

The main contribution is not a new neural architecture. The contribution is the task formulation, the controlled synthetic data methodology, the label-after-generation process, the reviewed evaluation splits, the comparison of multiple model families, and the error analysis that explains where the supervised encoder succeeds and fails. The project therefore follows the course emphasis on novelty through task design, synthetic data quality, model comparison, tables/graphs, and misclassification analysis.

The final results show that the supervised encoder is competitive on the reviewed test split, reaching 0.6777 mean exact accuracy and 0.8862 weak-aspect F1. However, the encoder has a large OOD exact-accuracy drop, falling to 0.2879 on the OOD split. Zero-shot LLM prompting generalizes much better to OOD, reaching 0.6121 exact accuracy. The most stable and practically useful signal is weak-aspect detection: the encoder keeps weak-aspect F1 around 0.89 across dev, test, and OOD.

Report Scope and Submitted Artifacts

This report summarizes the final state of the GitHub repository and the committed experiment artifacts. The report does not introduce new experiments or unreported results. It uses the official dataset, baseline reports, encoder report, error-analysis outputs, and generated visuals in the repository.

Artifact group	Repository location	Purpose
Dataset	synthetic_interview_data/data/processed/ and data/reviewed/	Training split and project-team reviewed dev/test/OOD evaluation files
Rubric and schema	docs/label_schema.md; docs/rubric.md	Definition of the six-aspect ordinal scoring task
Baseline results	synthetic_interview_data/data/reports/ baseline_results.*	Majority and TF-IDF logistic regression baselines
LLM results	synthetic_interview_data/data/reports/ llm_baseline_results.*	Zero-shot and few-shot LLM baselines
Encoder results	synthetic_interview_data/data/reports/ encoder_baseline_results.*	Supervised DistilBERT encoder baseline
Error analysis	synthetic_interview_data/data/reports/ error_analysis_results.*	Failure patterns, OOD drop analysis, and top error examples
Visualizations	synthetic_interview_data/data/visuals/	Final figures and summary tables for report/presentation

1. Motivation and Problem Context

Junior software developer interviews often ask candidates to explain a project they built or contributed to. A strong answer usually contains concrete technical details, clear personal ownership, an understandable explanation, a problem-solving process, a meaningful outcome, and relevance to junior software development work. Weak answers may be vague, overly team-focused, unclear, or unrelated to actual implementation.

This project turns that qualitative assessment into a measurable NLP task. Instead of asking an LLM to give generic interview advice, the project defines a structured prediction problem: convert a free-text candidate answer into six rubric-based ordinal scores. The result is a course-facing NLP/LLM experiment with a reproducible dataset, multiple baselines, a supervised encoder, OOD evaluation, error analysis, and final visualizations.

2. Formal Task Definition

Input: a free-text answer to a junior developer interview question about a project experience.

Output: a structured label object containing six ordinal scores from 1 to 5, plus derived weak and strong aspect lists.

Aspect	Score range	Definition
technical_depth	1-5	Technical reasoning, implementation detail, debugging, tradeoffs, constraints, or meaningful tool use.
personal_contribution	1-5	How clearly the candidate explains their own work rather than only the team's work.
clarity	1-5	Organization, coherence, concreteness, and interview-readiness.
problem_solving	1-5	Challenge, reasoning, troubleshooting, decisions, iteration, or solution quality.
impact	1-5	Outcome, value, delivery, users, metrics, improvement, or meaningful learning.
role_relevance	1-5	Fit to the interview question and to junior software-development work.

Derived labels are computed deterministically from final_scores: weak_aspects are all aspects with score <= 2; strong_aspects are all aspects with score >= 4.

3. Project Novelty

The project's novelty is the formulation and methodology rather than a new model architecture. A simple version of this project could have been a binary classifier for 'good' versus 'bad' answers. Instead, the project defines a multi-aspect ordinal scoring task that resembles aspect-oriented evaluation: a single answer can be technically strong but unclear, clear but low impact, or relevant but weak in personal contribution.

The synthetic data is also generated and validated as a controlled experimental artifact rather than as a single GPT batch. The pipeline uses profile-conditioned generation, label-after-generation scoring, independent validation, filtering, grouped splitting, and project-team review of evaluation splits. This aligns with the course preference for controlled synthetic data, multiple baselines, and evaluation insight rather than a simple LLM application.

4. Dataset Construction

4.1 Official Dataset Version

The frozen official dataset version is official_v1. It contains accepted synthetic training data and project-team reviewed evaluation files.

Item	Count / status
Total accepted records	558
Training split	265
Project-team reviewed dev split	64
Project-team reviewed test split	106
Project-team reviewed OOD split	55
Final reviewed evaluation records	225
Duplicate IDs	0
Duplicate answers	0
Train/test leakage	0
OOD/train leakage	0
Accepted labeler-validator delta >= 2	0

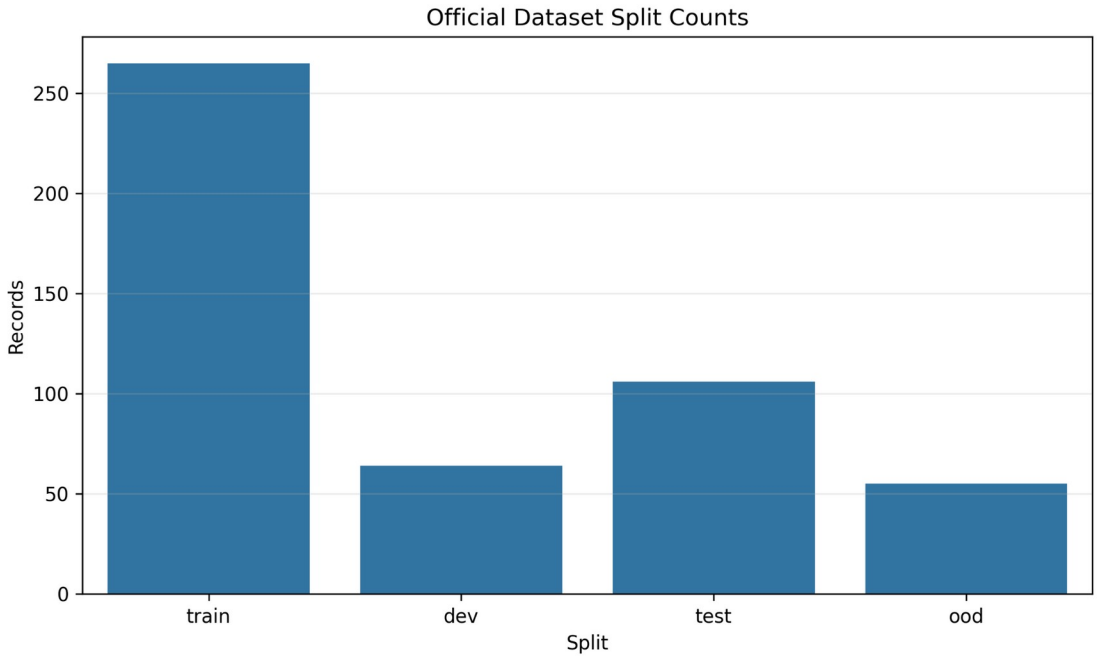

Figure 1. Official dataset split sizes for official_v1. Training uses synthetic accepted records; final evaluation uses project-team reviewed dev/test/OOD splits.

4.2 Generation Sources and Coverage

The dataset was built through several generation phases. The broad phase created normal mixed-quality answers. Targeted patches were then added to improve weak-label coverage, strong-answer coverage, high-impact examples, and rare-label diversity.

Generation source	Purpose	Clean accepted records
broad	Normal mixed junior answers	270
weak_patch	Targeted weak-label coverage	175
strong_patch	High-quality answer coverage	40
high_impact_patch	High-impact coverage repair	35
diversity_patch	Rare and missing-label coverage repair	38

Score coverage in official_v1 after generation and filtering.

Aspect	Low (1-2)	Mid (3)	High (4-5)
technical_depth	236	134	188
personal_contribution	269	44	245
clarity	44	97	417
problem_solving	279	93	186
impact	409	50	99
role_relevance	137	75	346

4.3 Reviewed Evaluation Data

The original dev/test/OOD files were preserved as synthetic review-candidate sources. The final evaluation uses project-team reviewed dev, test, and OOD files. The reviewed labels were checked with the project rubric; they are suitable for course-level evaluation but are not external expert annotations.

5. Data Methodology and Validation

The core design decision is label-after-generation. The pipeline first generates an answer from a controlled profile, then scores the generated answer using a rubric-based labeler, and finally applies an independent validator. Labels are based on visible evidence in the answer rather than copied from the intended profile.

Pipeline step	Role in quality control
Controlled profile sampling	Defines scenario, answer style, quality target, domain, and candidate behavior.
Answer generation	Produces natural interview-style answers under the controlled profile.
Rubric labeler	Assigns six aspect scores from evidence in the generated answer.
Independent validator	Rescores and audits the answer independently.
Agreement and evidence checks	Rejects or routes examples with leakage, contradictions, profile mismatch, or large disagreement.
Grouped split	Separates train/dev/test/OOD while checking scenario leakage.
Project-team review	Creates final reviewed dev/test/OOD evaluation sets.

Accepted examples must have all six final scores, valid weak/strong aspect derivations, no accepted score delta of 2 or more between labeler and validator, no label leakage in the answer text, and no train/test or OOD/train scenario leakage.

6. Models and Experimental Setup

Model / method	Input features	Training / prompting	Role in experiment
Majority baseline	None beyond training labels	Predicts the most common training score per aspect	Sanity baseline
TF-IDF logistic regression	Answer text only	TF-IDF features with one logistic regression classifier per aspect	Classical ML baseline
Zero-shot LLM	Question/answer and rubric prompt	No training examples in prompt	Off-the-shelf LLM baseline
Few-shot LLM	Question/answer, rubric, and 3 train examples	Deterministic few-shot selection from train	Off-the-shelf LLM baseline with examples
Supervised encoder	Answer text only	distilbert-base-uncased; shared encoder with six five-class heads	Fine-tuned/supervised baseline

6.1 Supervised Encoder Training Configuration

Parameter	Value
Base model	distilbert-base-uncased
Formulation	One shared encoder, one five-class classification head per rubric aspect
Input features	answer text only
Labels	final_scores only
Score mapping	scores 1-5 mapped to classes 0-4 during training
Epochs	3
Batch size	8
Learning rate	2e-5
Max sequence length	256
Seed	42
Device	CPU in the committed run

7. Evaluation Metrics

The task has six ordinal outputs, so the evaluation uses more than accuracy. Exact scoring is important, but ordinal distance, low/mid/high bands, weak-aspect detection, OOD performance, and qualitative error patterns are also necessary.

Metric	What it measures	Why it is useful
Mean exact accuracy	Average exact score accuracy across six aspects	Main strict score metric
Macro-F1 / weighted-F1	Per-aspect class balance and label performance	Handles uneven score distributions
Low/mid/high macro-F1	Band-level performance for weak/acceptable/strong groups	More robust than exact 1-5 boundaries
Mean absolute error (MAE)	Average ordinal score distance	Distinguishes one-level errors from severe errors
Weak-aspect precision/recall/F1	Detection of aspects with score <= 2	Practical for feedback and improvement suggestions
Confusion patterns	Common true-to-predicted score mistakes	Explains systematic failures
OOD drop	Test-to-OOD performance change	Measures generalization under distribution shift

8. Results

The final comparison includes five methods: majority, TF-IDF logistic regression, zero-shot LLM, few-shot LLM, and supervised encoder. The most important final evaluation splits are reviewed test and reviewed OOD. Dev is included for completeness.

Table 1. Aggregate model comparison across dev/test/OOD.

Model	Split	Exact accuracy	Low/mid/high F1	MAE	Weak F1
Majority	dev	0.3411	0.2372	0.9453	0.4667
Majority	test	0.2830	0.1852	1.1384	0.4422
Majority	OOD	0.2242	0.0956	1.9424	0.3333
TF-IDF logistic regression	dev	0.7318	0.6036	0.2917	0.8121
TF-IDF logistic regression	test	0.6038	0.5020	0.4182	0.7646
TF-IDF logistic regression	OOD	0.3061	0.2621	0.7848	0.8636
Zero-shot LLM	dev	0.7708	0.7150	0.2474	0.9024
Zero-shot LLM	test	0.6698	0.5789	0.3318	0.8163
Zero-shot LLM	OOD	0.6121	0.2792	0.4182	0.8753
Few-shot LLM	dev	0.7448	0.7193	0.2812	0.9412
Few-shot LLM	test	0.6352	0.5808	0.3664	0.8092
Few-shot LLM	OOD	0.5848	0.2780	0.5121	0.8518
Supervised encoder	dev	0.6823	0.6652	0.3516	0.8989
Supervised encoder	test	0.6777	0.5721	0.3428	0.8862
Supervised encoder	OOD	0.2879	0.2560	0.7152	0.8889

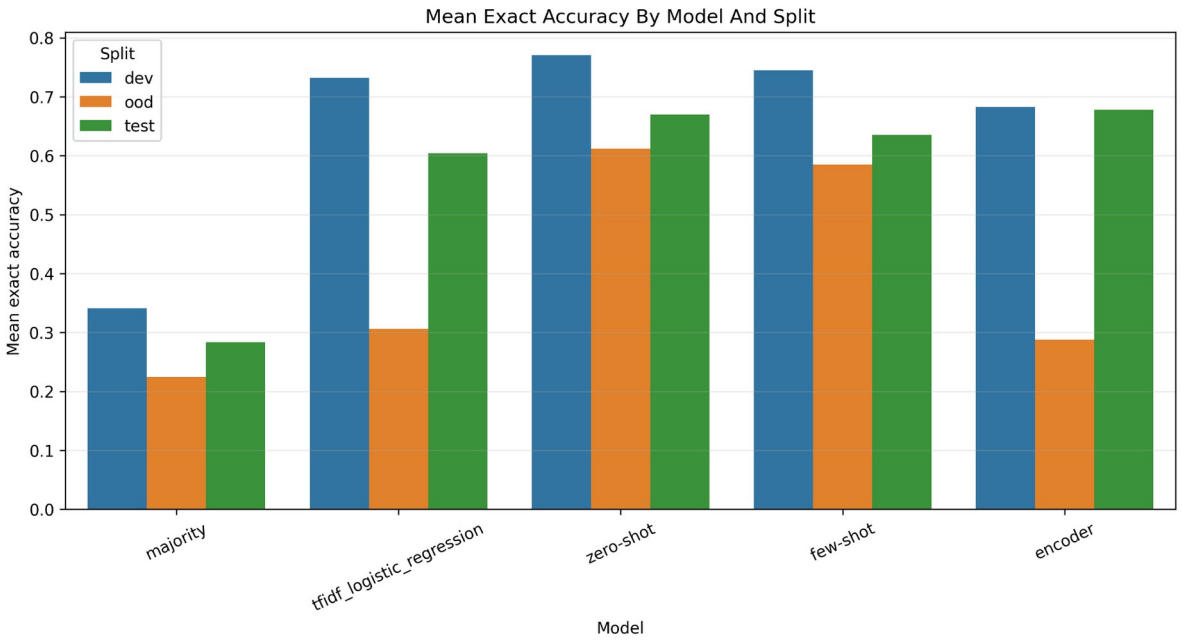

Figure 2. Mean exact accuracy by model and split. The supervised encoder is competitive on reviewed test, while zero-shot LLM is much stronger on OOD.

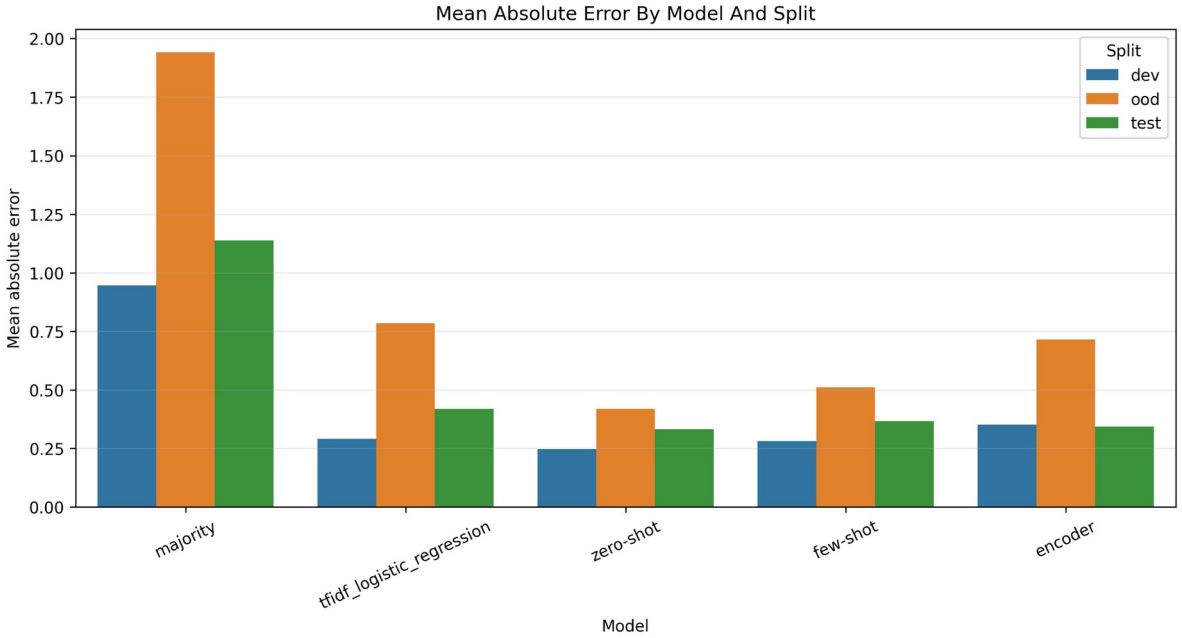

Figure 3. Mean absolute error by model and split. Lower is better; zero-shot LLM has the strongest OOD MAE among the non-majority methods.

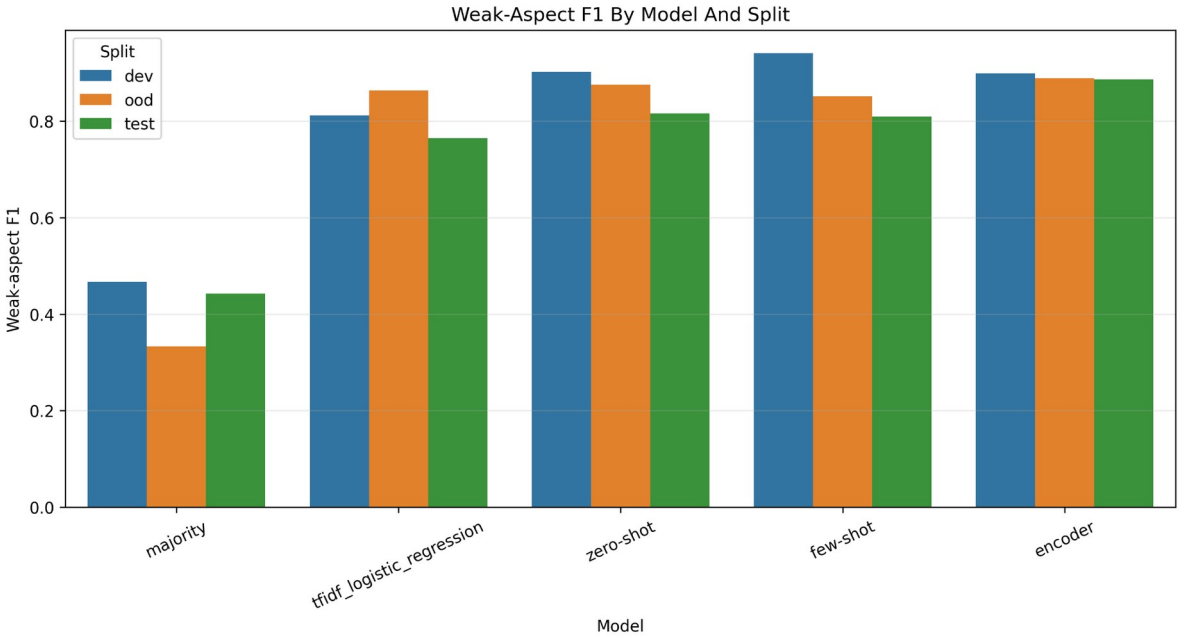

Figure 4. Weak-aspect F1 by model and split. Weak-aspect detection remains strong even when exact OOD scoring is difficult.

8.1 Main Result Interpretation

The supervised encoder reaches 0.6777 mean exact accuracy on the reviewed test split, slightly above the zero-shot LLM's 0.6698 and above TF-IDF's 0.6038. However, this advantage does not transfer to OOD. On the OOD split, the encoder drops to 0.2879 exact accuracy, while zero-shot LLM remains at 0.6121.

The encoder's strongest practical behavior is weak-aspect detection. It achieves weak-aspect F1 of 0.8989 on dev, 0.8862 on test, and 0.8889 on OOD. This indicates that exact 1-5 scoring is harder than detecting whether an answer has weak aspects.

TF-IDF is a surprisingly strong classical baseline. It beats the encoder on dev exact accuracy and slightly beats it on OOD exact accuracy, but the encoder is stronger on reviewed test exact accuracy and weak-aspect F1. The majority baseline is consistently weak and functions mainly as a sanity baseline.

9. Supervised Encoder Analysis

Table 2. Encoder split-level error summary.

Split	Records	Exact accuracy	MAE	Exact-all rate	Severe error rate	Weak precision	Weak recall	Weak F1
dev	64	0.6823	0.3516	0.1250	0.0339	0.8696	0.9302	0.8989
test	106	0.6777	0.3428	0.0849	0.0189	0.8529	0.9223	0.8862
OOD	55	0.2879	0.7152	0.0000	0.0030	1.0000	0.8000	0.8889

The encoder is reasonably accurate on dev and test, but its exact-all-aspects rate is low: 12.5% on dev and 8.49% on test. This means that even when per-aspect metrics look competitive, most answers still have at least one aspect-level mistake. On OOD, the exact-all-aspects rate is 0.0%.

The severe error rate is low across all splits, especially on OOD. This shows that many encoder mistakes are boundary errors, not extreme misclassifications. This is why MAE and low/mid/high metrics are essential alongside exact accuracy.

10. OOD Generalization

Table 3. OOD deltas relative to reviewed test. Negative exact-accuracy deltas indicate an OOD performance drop.

Model	Exact accuracy delta (OOD - test)	MAE delta (OOD - test)	Weak F1 delta (OOD - test)
Majority	-0.0588	0.8040	-0.1089
TF-IDF logistic regression	-0.2977	0.3666	0.0990
Zero-shot LLM	-0.0577	0.0864	0.0590
Few-shot LLM	-0.0504	0.1457	0.0426
Supervised encoder	-0.3898	0.3724	0.0027

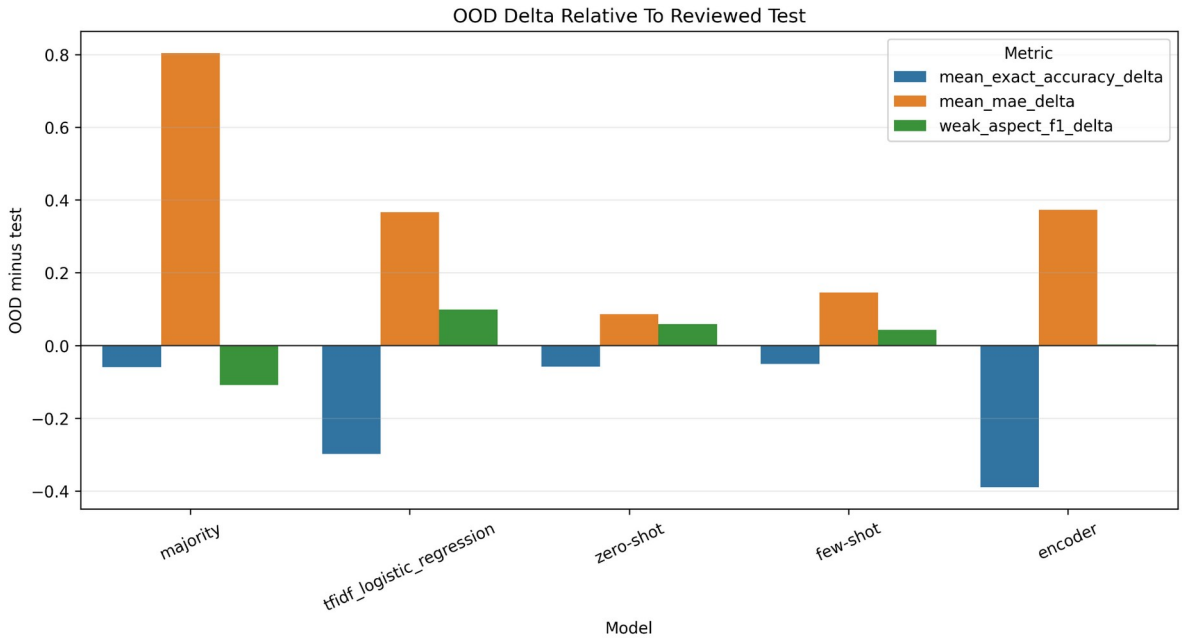

Figure 5. OOD metric deltas relative to reviewed test. The encoder has the largest exact-accuracy drop, while zero-shot and few-shot LLMs are much more stable.

The encoder has the largest exact-accuracy OOD drop: -0.3898. In comparison, zero-shot LLM drops only -0.0577 and few-shot drops -0.0504. This is the clearest evidence that the supervised encoder learned regular synthetic patterns but did not generalize as well to OOD examples.

The OOD drop is not only a failure; it is a key experimental finding. It shows why an OOD split was necessary. If the project only reported reviewed test performance, the encoder would appear competitive with the LLM baselines. The OOD split reveals that zero-shot LLM prompting is more robust under distribution shift.

11. Per-Aspect Error Analysis

Table 4. Encoder per-aspect failure patterns across all evaluation records.

Aspect	Exact accuracy	MAE	Signed error	Bias direction	Severe error rate	Top confusions
Technical depth	0.5067	0.4933	0.1378	overpredicts	0.0000	1->2 (55), 5->4 (24), 3->2 (16)
Personal contribution	0.6267	0.4089	0.2044	overpredicts	0.0356	1->2 (35), 3->4 (13), 3->2 (12)
Clarity	0.5333	0.4667	-0.4311	underpredicts	0.0000	4->3 (72), 5->4 (29), 3->4 (4)
Problem solving	0.6356	0.3778	0.0044	balanced	0.0133	1->2 (42), 5->4 (26), 3->2 (8)
Impact	0.6800	0.3689	-0.3333	underpredicts	0.0444	2->1 (40), 3->2 (9), 5->4 (8)
Role relevance	0.5200	0.5022	0.4844	overpredicts	0.0222	2->3 (54), 3->4 (47), 2->4 (3)

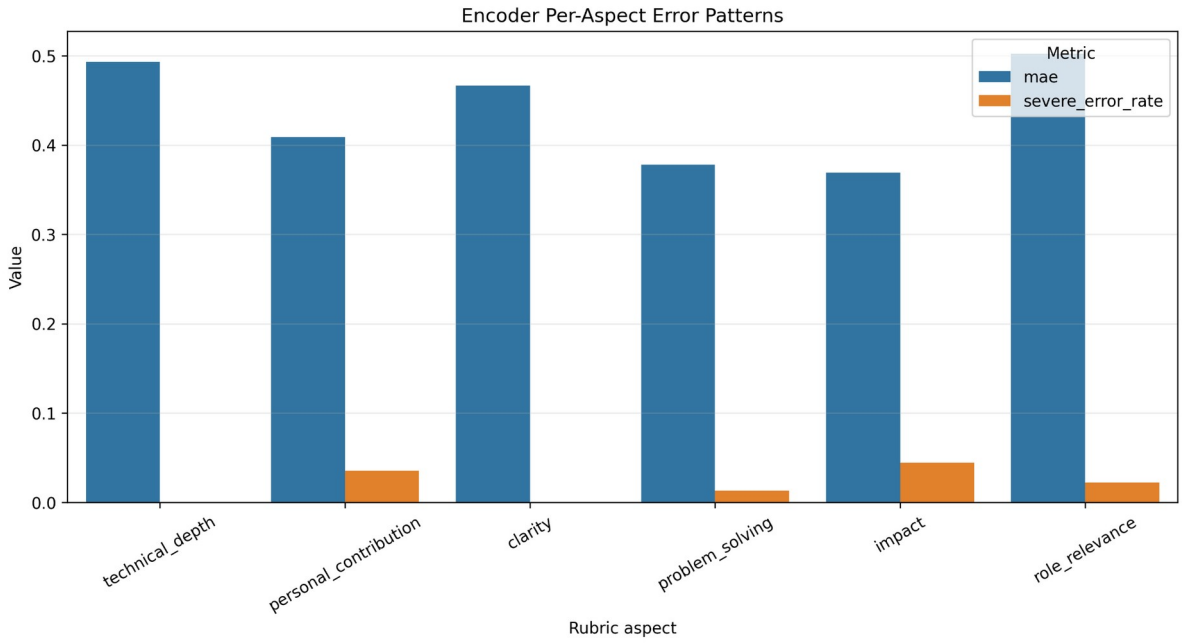

Figure 6. Encoder per-aspect error patterns. Role relevance, technical depth, and clarity have the highest MAE; severe errors remain relatively rare.

11.1 Aspect-Level Findings

Role relevance is the hardest aspect overall by MAE. The encoder strongly overpredicts it, with common confusions such as 2->3 and 3->4. This is especially important for OOD, where many examples are weakly related to software development but are predicted as more relevant than the rubric labels indicate.

Technical depth also has high MAE. On OOD, all 55 technical-depth examples follow the same confusion pattern: true 1 -> predicted 2. The model recognizes that the answer is weak, but it softens the score rather than assigning the lowest label.

Clarity tends to be underpredicted. The most common clarity confusion is 4->3, indicating that the model often downgrades otherwise clear answers by one level. Impact also tends to be underpredicted, with common 2->1 and 3->2 confusions and a small number of high-impact examples incorrectly flagged as weak.

Problem solving is relatively balanced, with near-zero signed error and lower MAE than the hardest aspects. It is also the best OOD aspect for the encoder. This suggests that problem-solving cues may be easier to detect from explicit challenge/action/fix language.

12. Weak-Aspect Detection

Weak-aspect detection is the most stable and practically useful behavior in the project. The encoder keeps high weak-aspect F1 across dev, test, and OOD, even when exact OOD score accuracy drops. On OOD, the encoder's weak-aspect precision is 1.0 and recall is 0.8, meaning predicted weak aspects are reliable but some true weak aspects are missed.

This result is important because interview feedback often needs to identify what to improve more than it needs an exact 1-5 score. A model that reliably identifies weak technical depth, unclear personal contribution, or weak impact can still support useful feedback even when exact ordinal scoring is imperfect.

13. Representative Error Examples

The top encoder error examples show two major error families. First, high-impact or strong examples can be underpredicted, especially on impact. For example, top test errors include cases where impact is falsely predicted as weak. Second, OOD role relevance is often missed as weak: many OOD examples have role_relevance as a missed weak aspect.

Many top OOD examples have high total absolute error but max per-aspect error of only 1. This means the model is often wrong across many aspects by one score point, rather than producing one severe outlier. This supports the interpretation that many errors are ordinal-boundary mistakes.

Error pattern	Evidence from error analysis	Interpretation
False weak impact	Top examples include strong/high-impact test cases with false weak impact.	The encoder can under-credit concrete outcomes.
Missed weak role relevance	Many OOD top errors list role_relevance as a missed weak aspect.	The encoder overestimates relevance of non-software or weakly relevant answers.
Boundary errors	Several top OOD cases have total error 5-6 but max error 1.	Exact scoring fails while ordinal distance remains modest.
High-score compression	Strong examples often receive 4 instead of 5 across several aspects.	The encoder is conservative at the top end of the rubric.

14. Final Visual Evidence

The final visuals in data/visuals support the report narrative. The exact-accuracy figure communicates the model comparison, the MAE figure captures ordinal distance, the weak-F1 figure highlights practical robustness, the OOD-delta figure explains generalization failure, and the per-aspect error figure connects the quantitative results to the error analysis.

Figure	Use in report	Main message
model_comparison_mean_exact.png	Main results	Encoder is competitive on test; zero-shot LLM is strongest on OOD.
model_comparison_mae.png	Ordinal evaluation	MAE shows score-distance quality; lower is better.
weak_aspect_f1_by_model.png	Practical usefulness	Weak-aspect detection remains strong across splits.
ood_drop_by_model.png	Generalization	Encoder has the largest exact-accuracy OOD drop.
encoder_per_aspect_errors.png	Error analysis	Role relevance, technical depth, and clarity are hard aspects.
dataset_split_counts.png	Dataset section	Shows official split sizes and project-team reviewed evaluation data.

15. Discussion

The results support three main conclusions. First, a controlled synthetic dataset can train a supervised encoder that performs competitively on regular reviewed evaluation data. Second, exact six-aspect ordinal scoring remains difficult, especially under OOD shift. Third, weak-aspect detection is more robust than exact scoring and is likely the most useful output for feedback-oriented applications.

The model comparison is central to the project. The majority baseline confirms that the task is not solved by always predicting frequent labels. TF-IDF shows that shallow lexical cues are surprisingly informative. The supervised encoder learns useful patterns and reaches the best reviewed test exact accuracy among the non-LLM and LLM baselines in the

committed comparison. Zero-shot LLM, however, is much more robust on OOD, suggesting that general language-model reasoning transfers better than a small encoder trained only on the synthetic training split.

The error analysis is also central. Without it, the OOD drop would be only a number. The analysis shows that many failures are systematic: role relevance is overpredicted, clarity and impact are underpredicted, high scores are compressed downward, and weak OOD examples are often predicted as slightly less weak. These findings connect model behavior to the rubric and to the synthetic data design.

16. Limitations and Caveats

Limitation	Effect on interpretation
Synthetic data	Generated answers may contain style artifacts or simplified patterns that differ from real interviews.
Project-team reviewed labels	Evaluation labels are reviewed with the project rubric but are not external expert annotations.
Subjective ordinal scores	Adjacent 1-5 labels can be ambiguous, especially for impact and role relevance.
Small OOD split	OOD analysis is meaningful but based on 55 reviewed examples.
Answer-only encoder	The supervised encoder intentionally uses only answer text, not metadata or full interview context.
Baseline training run	The encoder is a baseline model, not a production hiring system.

These limitations do not invalidate the project. They define the appropriate scope of the conclusions: the project demonstrates a well-framed course-level NLP experiment with controlled synthetic data, reviewed evaluation splits, multiple baselines, and interpretable error analysis.

17. Alignment with Course Requirements

Course requirement / expectation	Evidence in this project
Novel and relevant task	Fine-grained scoring of junior developer interview answers with a six-aspect rubric.
Clear input/output definition	Free-text answer input; six ordinal scores plus weak/strong aspect labels as output.
Synthetic training/evaluation data	official_v1 synthetic dataset with controlled generation phases and reviewed evaluation splits.
Data generation methodology	Controlled profiles, label-after-generation scoring, validation, filtering, and leakage-safe splitting.
Several model comparisons	Majority, TF-IDF, zero-shot LLM, few-shot LLM, and supervised encoder.
Fine-tuned/supervised model	DistilBERT encoder with six classification heads trained on train.jsonl.
Off-the-shelf model comparison	Zero-shot and few-shot LLM baselines.
Metrics beyond accuracy	MAE, low/mid/high macro-F1, weak-aspect F1, confusion patterns, OOD deltas.
Tables and graphs	Final visualizations and comparison/error tables under data/visuals and data/reports.
Error analysis	Per-aspect failure patterns, top error examples, weak-aspect behavior, and OOD discussion.
GitHub repository artifacts	Code, data, reports, visuals, README files, and reproducible scripts are committed.
Team members	Daniel Shatzov and Shai Gigi listed in the root README.

18. Conclusion

Junior Interview Answer Scoring is a complete course-level NLP/LLM project. It defines a realistic and measurable task, builds a controlled synthetic dataset, validates labels through labeler/validator agreement and project-team review, compares multiple baseline families, trains a supervised encoder, evaluates OOD generalization, and analyzes errors.

The final results show that the supervised encoder is useful on regular reviewed evaluation data and strong for weak-aspect detection, but it does not generalize as well as zero-shot LLM on OOD examples. This is the central insight of the project: model comparison and OOD evaluation reveal behavior that a single test score would hide.

The project therefore satisfies the core course goals: a novel task framing, synthetic data generation, model training, comparison with off-the-shelf models, serious evaluation, visual results, and error analysis that explains what the model learns and where it breaks.

Appendix A. Reproducibility Commands

Run tests:

```
cd synthetic_interview_data
python3 -m unittest discover -s tests -v
```

Run classical baselines:

```
cd synthetic_interview_data
python3 scripts/run_baselines.py
```

LLM dry-run smoke test:

```
cd synthetic_interview_data
python3 scripts/run_llm_baselines.py --mode all --splits dev --dry-run --limit 5 --output-
dir /private/tmp/llm_baseline_smoke
```

Encoder smoke test:

```
cd synthetic_interview_data
python3 scripts/run_encoder_baseline.py --model-name distilbert-base-uncased --epochs 1 --
batch-size 4 --limit-train 20 --limit-eval 10 --output-dir /tmp/encoder_baseline_smoke
```

Error analysis:

```
cd synthetic_interview_data
python3 scripts/run_error_analysis.py --output-dir data/reports --top-n-examples 20
```

Final visuals:

```
cd synthetic_interview_data
python3 scripts/make_final_visuals.py --output-dir data/visuals --format png
```